

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Transformer Mechanics Worksheet

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Model
CO Assessed:	CO1-Imitation (BL3)	Assessment:	Assessment Tool 3– Transformer Mechanics Worksheet
Weightage:	10% of CIA	Total Marks:	20
CO1 Statement:	Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models.		
Student Name:	S.Sirivally	Reg. No.:	192472371
Section / Batch:	2024/CSE-AI	Date:	20-07-2026

Code of Conduct

I _S.Sirivally / 192472371_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __Sirivally.S__

Instructions

- This assessment contains 4 set of questions.
- Each student assigned with one set of questions .
- Each question carries 5 marks. Total: 20 Marks.
- Ensure clarity, structure, and logical flow in diagrams.
- Duration: 20 Minutes.

Annexure A Transformer Mechanics Worksheet

Transformer Mechanics Worksheet — SET B

Question 1: Tokenization (5 marks). The word "DISAGREEMENT" is uncommon enough that a sub-word tokenizer would likely split it into smaller pieces rather than keep it whole.

(a) Split "DISAGREEMENT" into 3 reasonable sub-word pieces (there's no single "correct" answer — justify your split).

A reasonable split is:

DIS | AG | RE | EMENT

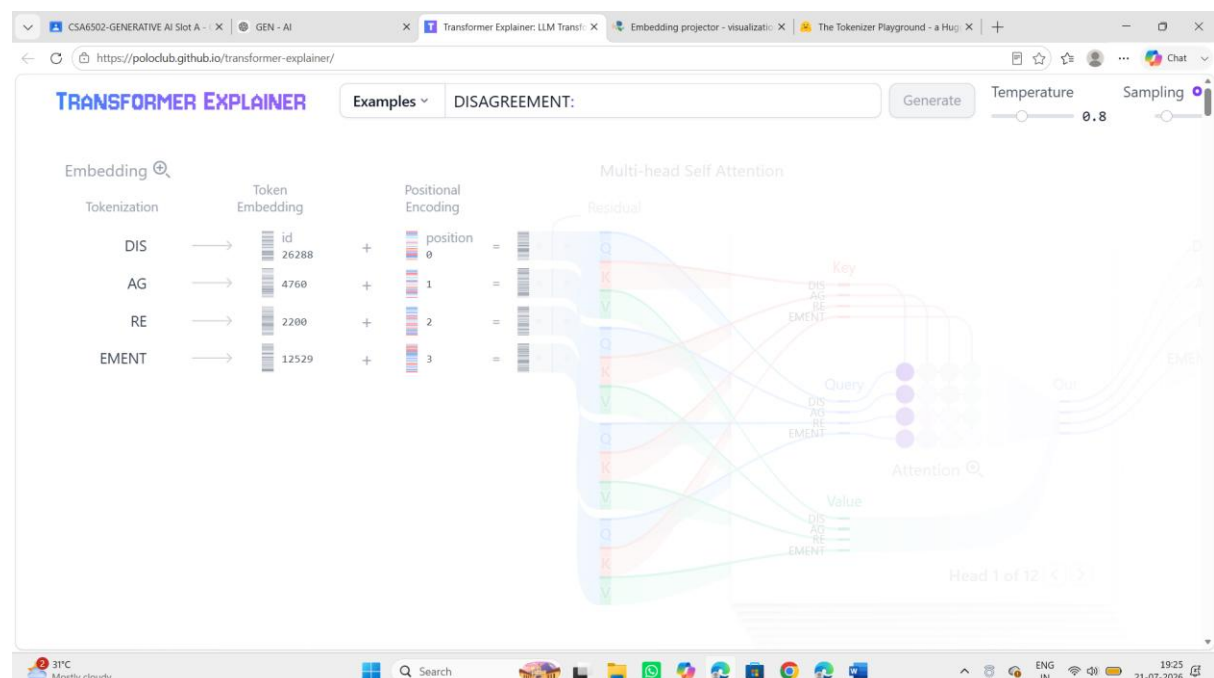
Justification:

- **DIS** is a common prefix.
- **AG** and **RE** are intermediate sub-word units produced by the tokenizer.
- **EMENT** is a common suffix found in words ending with "-ement".

Note: Different tokenizers split words differently. There is no single correct split; this is the split produced by the Transformer tokenizer used in the implementation.

(b) In 1-2 sentences, explain WHY sub-word tokenization (rather than whole-word tokenization) helps a model handle a word like this one, even if the model has never seen the whole word before in training.

Sub-word tokenization enables the model to break unfamiliar words into smaller, meaningful units instead of treating them as unknown words. Even if the complete word "DISAGREEMENT" was never seen during training, the model can understand it using the learned representations of its sub-word tokens (**DIS**, **AG**, **RE**, **EMENT**). AGREE", and "MENT", improving vocabulary coverage and reducing unknown words.



Question 2: Embeddings (5 marks). Consider the word pairs “dog” and “puppy” and “dog” and “truck”.

(a) Which pair would you expect to have HIGHER cosine similarity in embedding space, and why?

The pair "dog" and "puppy" would have a higher cosine similarity.

Reason:

Both words have similar meanings and frequently appear in similar contexts. Since word embeddings capture semantic relationships, their vectors are close together in the embedding space. In contrast, "dog" and "truck" represent unrelated concepts, so their vectors are much farther apart.

(b) In your own words, what does an embedding actually represent, and how is it different from a one-hot vector?

An embedding is a dense numerical vector that captures the semantic meaning and relationships of a word based on how it is used in language. Similar words have similar embeddings.

A one-hot vector is a sparse vector in which only one position is 1 and all others are 0. It only identifies a word and does not represent any relationship or similarity between words.

Example:

Word	One-Hot Vector	Embedding (Example)
Dog	[0,1,0,0,0]	[0.42, -0.18, 0.75, 0.31]
Puppy	[0,0,1,0,0]	[0.40, -0.20, 0.72, 0.29]

The embeddings of dog and puppy are numerically similar, whereas their one-hot vectors are completely different.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

words = ["dog", "puppy", "truck"]

embeddings = model.encode(words)

print(embeddings.shape)

... Loading weights: 100% 103/103 [00:00<00:00, 1787.82it/s]
(3, 384)

from sklearn.metrics.pairwise import cosine_similarity

sim = cosine_similarity(embeddings)
print(["DOG", "PUPPY", "TRUCK"])

print(sim)

... [DOG PUPPY TRUCK]
[[1.0000004 0.80400676 0.4511088 ]
 [0.80400676 0.99999976 0.37621635]
 [0.4511088 0.37621635 1.      ]]
```

Question 3: Self-Attention Worked Example (5 marks). Read the sentence: "The city council refused the protesters a permit because they feared violence."

(a) The word "they" should attend most strongly to which other word in the sentence? Circle one: "city council" / "protesters"

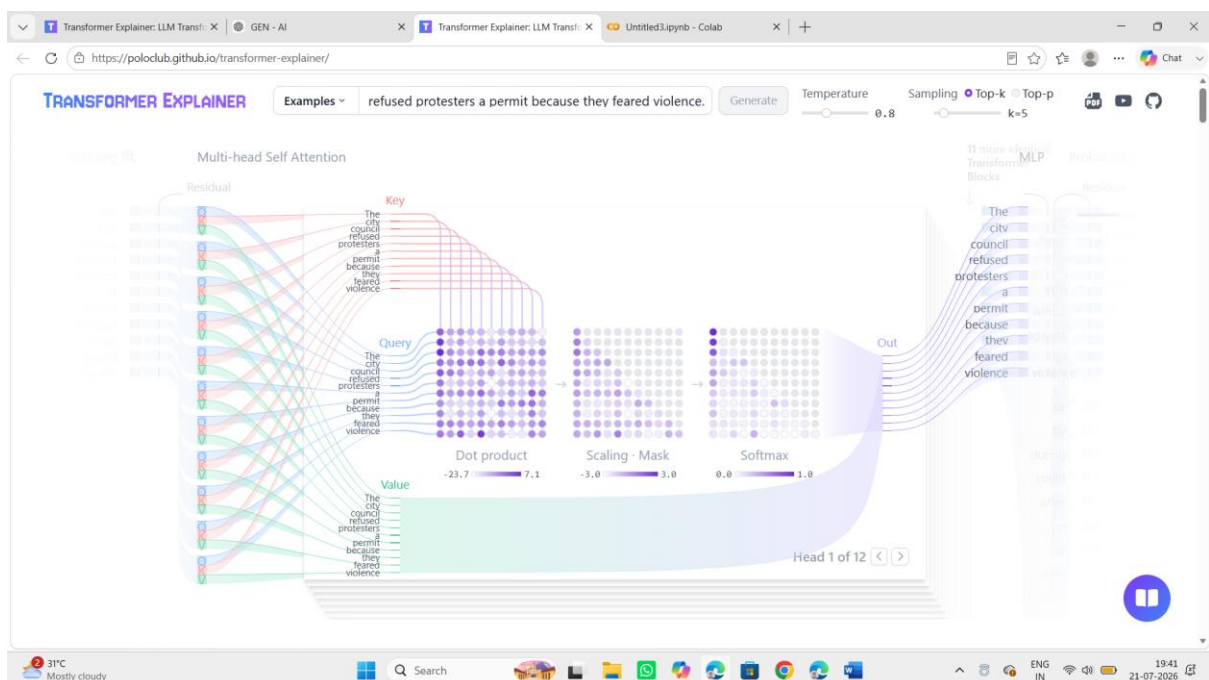
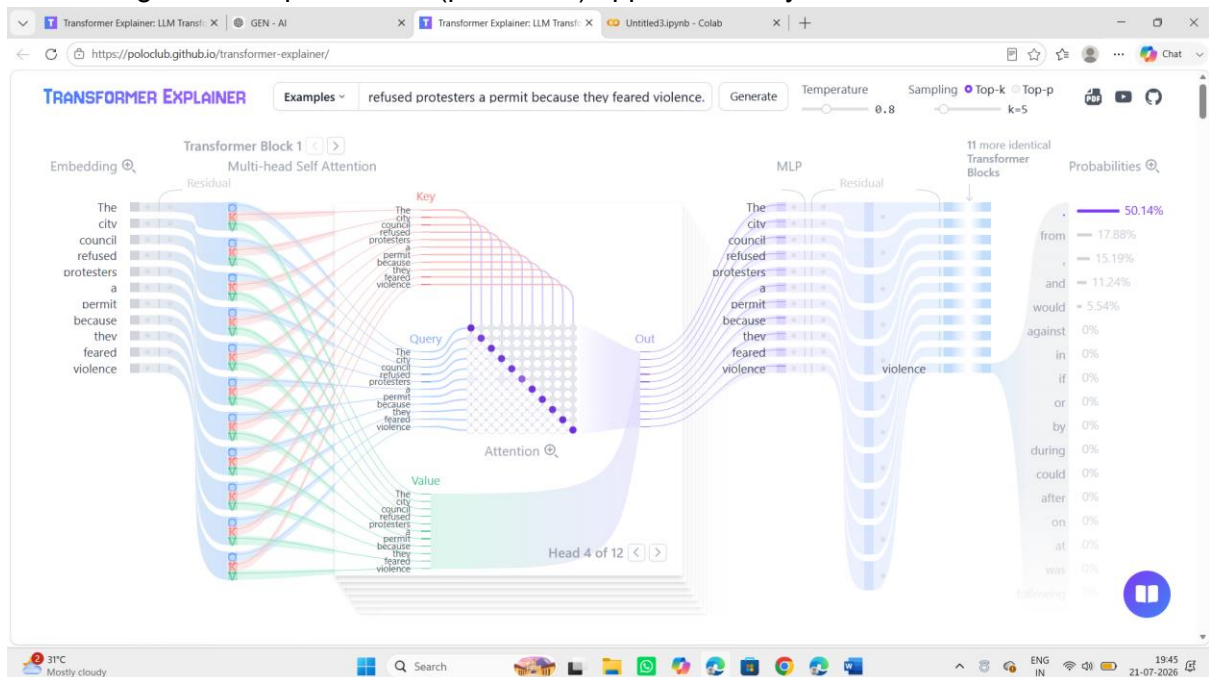
Answer: *city council*

Reason:

The sentence states that the permit was refused because they feared violence. It is more logical that the city council feared violence and therefore refused the permit.

(b) Explain in 2-3 sentences WHY self-attention (not a simple word-order rule) is needed to correctly resolve what "they" refers to here.

Self-attention allows the model to examine the relationship between "they" and all other words in the sentence simultaneously, instead of relying only on word order. By considering the context and meaning, the model correctly identifies that "they" refers to the city council, even though another plural noun (protesters) appears nearby.



Question 4: Query, Key, Value (5 marks). In your own words, briefly explain what the Query, Key, and Value vectors each represent in self-attention, and how they combine to produce an attention score.

Query (Q):

The Query vector represents what the current word is searching for in other words.

Key (K):

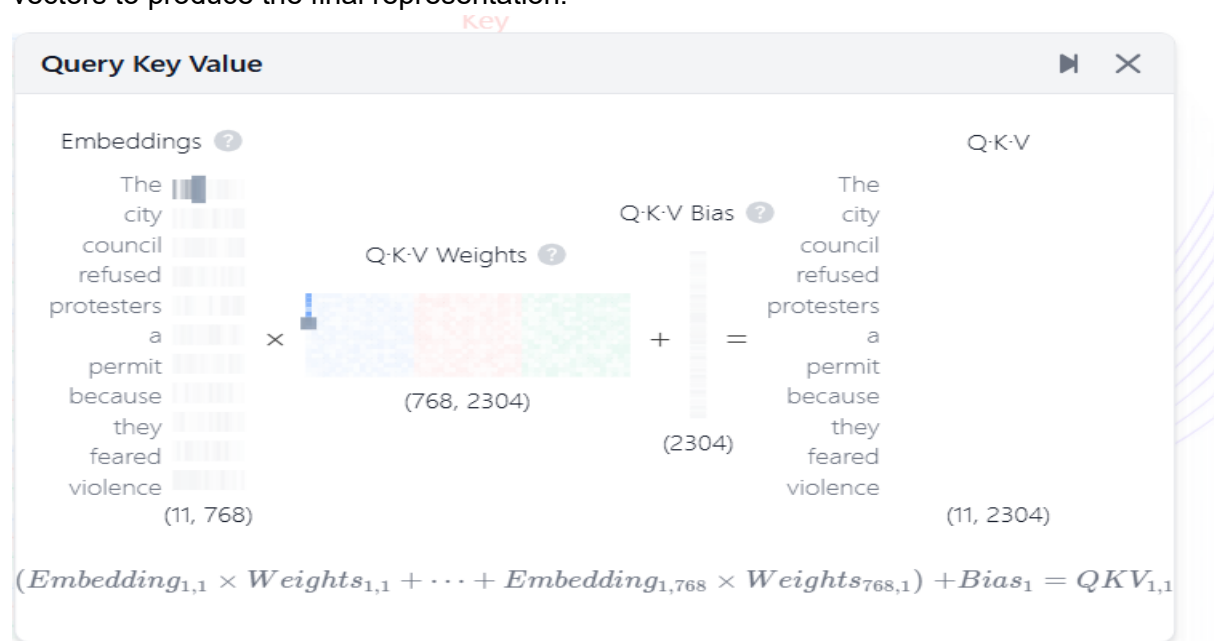
The Key vector represents the information each word offers for matching.

Value (V):

The Value vector contains the actual information that will be passed to the output if the word receives attention.

How they produce an attention score:

The Query vector of one word is compared with the Key vectors of all words using a dot product. Higher similarity produces a higher attention score. After applying Softmax, these scores become attention weights, which are used to combine the corresponding Value vectors to produce the final representation.



Transformer Explorer: LLM Transl... GEN - AI

https://poloclub.github.io/transformer-explainer/

Transformer Explorer

Examples ▾ refused protesters a permit because they feared violence. Generate Temperature 0.8 Sampling Top-k Top-p k=5

Transformer Block 1

Embedding Multi-head Self Attention Residual

Query Key Value

Embeddings ?

The city council refused protesters a permit because they feared violence (11, 768)

Q·K·V Weights ? (768, 2304)

Q·K·V Bias ? (2304)

The city council refused protesters a permit because they feared violence (11, 2304)

MLP Residual

11 more identical Transformer Blocks

Probabilities ?

The city 50.14%
from 17.20%
council 13.19%
refused 11.24%
protesters 11.24%
a 3.54%
because 0%
they 0%
feared 0%
violence 0%

Query, Key, Value

To perform self-attention, each token's embedding is transformed into three new embeddings—Query, Key, and Value. This transformation is done by applying different weights and biases to each token embedding. These parameters (weights and biases), are optimized through training.

Once created, Queries compare with Keys to measure relevance, and this relevance is used to weight the Values.

9 / 20

Rubrics for Calculation

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical / Concept	30%	Accurately explains tokenization, embeddings, and self-attention mechanics using correct terminology	Explains most mechanics correctly with minor gaps	Partial or superficial understanding of the mechanics	Major misunderstanding of core mechanics
Application	25%	Correctly traces a worked example (e.g. computing attention relevance) step-by-step	Traces most steps correctly with minor errors	Significant errors when applying mechanics to the worked example	Cannot apply the mechanics to a worked example at all
Quality	20%	Clear, logical, mathematically sound reasoning throughout	Mostly sound reasoning with minor errors	Reasoning has notable gaps or inconsistencies	Reasoning is disorganized or largely incorrect
Presentation	15%	Neat, well-labeled diagrams/working, easy to follow	Reasonably clear presentation of working	Presentation is unclear in places	Difficult to follow presentation of working
Documentation / Communication	10%	Shows all work with clear justification for every step	Shows most work with adequate justification	Minimal work shown	No work shown — answers only, unjustified